Lab 1: First steps with PyTorch

Tensors, a small neural network, and regression

Tutorial: 22 October 2026 **Duration:** 90 minutes

Goal. Represent observations as tensors, evaluate a small neural network, measure its prediction error, and see what changes during training. No previous experience with NumPy or PyTorch is assumed. Work in your own copy of `lab_01.ipynb` and run cells in order. **Supplied** cells provide examples, plotting, and training machinery; **Your turn** cells contain short tasks and space for explanations. All computations use the CPU.

1. Tensors and basic operations

20 minutes

A tensor stores numbers in an array. A vector has one dimension; a matrix has two. Begin with this **supplied** working example:

```
import torch
v = torch.tensor([-1., 0., 2.])
print(v)
print(v.shape)
print(v[0])
print(v**2)
```

The decimal points create floating-point numbers. Indexing starts at zero; `**2` squares each entry. For a matrix, `A.shape` reports its number of rows and columns.

`torch.tensor([1., 2.])`	Create a tensor from a list.
`u.shape`, `u[:3]`	Inspect dimensions; take the first three entries.
`u.sum()`, `u.mean()`	Sum or average all entries.
`u**2`, `A * B`	Square entries; multiply entry by entry.
`A @ B`, `A.T`	Matrix multiplication; transpose a matrix.
`torch.relu(u)`	Apply $\max\{t, 0\}$ to each entry.
`u.reshape(-1, 1)`	Make a column; `-1` infers its length.

Your turn.

(a) Create $u = (-2, -1, 0, 1, 2)$. Compute its sum, the mean of its squared entries, and its first three entries. Check the results by hand.

(b) Predict `torch.relu(u)`, then run it. Change one entry of u and explain the effect on the result.

(c) For $A = \left(\begin{smallmatrix} 1 & 2 \\ 0 & 1 \end{smallmatrix}\right)$ and $B = \left(\begin{smallmatrix} 1 & 0 \\ 3 & 2 \end{smallmatrix}\right)$, predict and compare `A * B` and `A @ B`. Create matrices using nested lists, for example `torch.tensor([[1., 2.], [0., 1.]])`.

(d) Run `column = u.reshape(-1, 1)`. Compare `u.shape` and `column.shape`. Why are the same five numbers now arranged differently?

From now on, a batch of n scalar inputs is a column tensor of shape $(n, 1)$: one observation per row, one input coordinate per observation.

From random numbers to observations

Generate and inspect a regression dataset

2. Generate the data

15 minutes

We observe independent pairs (X_i, Y_i) generated by

$$X \sim \mathrm{Unif}[-1, 1], \qquad Y = X^2 + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, 0.1^2),$$

where X and ε are independent. The noise has standard deviation 0.1, not variance 0.1. Our aim is to predict a new response from its input. First, we create observations ourselves so that we know the underlying relationship.

`torch.rand(n, 1)` generates a column of independent uniform numbers in $[0, 1)$. `torch.randn(n, 1)` generates independent standard normal numbers. Arithmetic acts on each entry. Setting a seed makes the numerical experiment reproducible.

Your turn.

(a) Complete the three expressions below. Use $n = 64$ observations.

```
torch.manual_seed(7)
n = 64
x = ...        # uniform inputs in [-1, 1]
noise = ...    # normal noise, standard deviation 0.1
y = ...        # observed responses
```

Hint. If U is uniform on $[0, 1]$, then $2U - 1$ is uniform on $[-1, 1]$. If Z is standard normal, then $0.1Z$ has the required noise distribution.

(b) Inspect the shapes of `x`, `noise`, and `y`. Print the first five inputs and responses using `x[:5]` and `y[:5]`. What does a row of the paired tensors represent?

(c) Run the **supplied** plotting cell, which displays the observations and the function $x \mapsto x^2$. Its essential commands are shown below. You may use this code without writing it yourself.

```
import matplotlib.pyplot as plt
grid = torch.linspace(-1, 1, 200).reshape(-1, 1)
plt.scatter(x[:, 0], y[:, 0], label="Observations")
plt.plot(grid[:, 0], grid[:, 0]**2,
         color="black", label="True regression function")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
```

Here `torch.linspace` creates equally spaced points; `x[:, 0]` selects the first column for plotting. The colon means all rows.

(d) Repeat the data and plot cells with noise standard deviation 0, then 0.3. Describe what changes. Which function lies at the centre of the observations, and why? Restore standard deviation 0.1 and seed 7 before continuing.

A neural network as a function

Translate matrix operations into PyTorch

3. Construct a small network

25 minutes

For one scalar input x, consider a network with two hidden neurons:

$$z = W_1 x + b_1, \qquad h = \mathrm{ReLU}(z), \qquad f_\theta(x) = W_2 h + b_2,$$

where ReLU acts on each coordinate and

$$W_1 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \quad b_1 = \begin{pmatrix} -0.5 \\ -0.5 \end{pmatrix}, \quad W_2 = \begin{pmatrix} 2 & -1 \end{pmatrix}, \quad b_2 = 0.5.$$

These numbers are the *parameters* of the network. We choose them by hand first; later, training will adjust parameters using observations.

Your turn.

(a) Compute z, h, and $f_\theta(1)$ on paper. Which hidden neuron is active, meaning that its value after ReLU is positive?

(b) Run the **supplied** parameter cell:

```
W1 = torch.tensor([[1.], [-1.]])
b1 = torch.tensor([-0.5, -0.5])
W2 = torch.tensor([[2., -1.]])
b2 = torch.tensor([0.5])
```

Our batch tensor stores inputs in rows, whereas the formula above uses column vectors. The corresponding batch formulas are

$$Z = \texttt{x @ W1.T + b1}, \quad H = \mathrm{ReLU}(Z), \quad \widehat{Y} = \texttt{H @ W2.T + b2}.$$

PyTorch adds the bias vector to each row automatically; this is called *broadcasting*. Complete the three expressions:

```
def predict(x):
    z = ...
    h = ...
    prediction = ...
    return prediction
```

The indented lines form a function; `return` gives back its result. Calling `predict(x)` reuses these operations with the current values of `W1`, `b1`, `W2`, and `b2`.

(c) Check your hand calculation using `predict(torch.tensor([[1.]]))`.
Then use `torch.tensor([[-1.], [0.], [1.]])`. Inspect the hidden tensor and output shapes. What does each row represent?

(d) Run the **supplied** network plot. Change the first hidden bias from -0.5 to 0. Where does the corresponding kink move? Restore the original bias afterwards.

(e) Temporarily replace `torch.relu(z)` by `z` in your function and plot again. Explain why the resulting function is affine. Restore ReLU before continuing.

Measure error and run training

Use familiar expressions inside supplied code

4. Mean squared error

10 minutes

For predictions $\widehat{y}_i$ and observed responses y_i, define

$$L_n = \frac{1}{n}\sum_{i=1}^{n}(\widehat{y}_i - y_i)^2.$$

This is the mean squared error (MSE), the loss used in this lab. **Your turn.**

(a) Calculate the MSE by hand for predictions $(2, 1)$ and targets $(1, 3)$. Reproduce your answer using tensors, subtraction, squaring, and `.mean()`.

(b) Run `prediction = predict(x)`. Check that `prediction.shape` and `y.shape` are both $(64, 1)$, then complete:

```
loss = ...
print(loss)
```

Hint. Apply the formula entry by entry and take the mean. Keep predictions and targets as columns of the same shape.

(c) Change one parameter and compare the loss. Why might improvement at one observation fail to reduce the whole loss? Restore the original parameter.

5. Train the network with supplied code

20 minutes

Training adjusts parameters to reduce the training loss. The notebook initializes a fresh network with 16 hidden neurons; your `predict` function works with these new tensor sizes. Initialization, the optimizer, evaluation, and plots are **supplied**. The optimizer uses loss derivatives to update parameters, as we will study in the optimization lab.

Your turn.

(a) Complete only the prediction and loss expressions in the supplied training cell. This is the structure of its loop:

```
for step in range(number_of_steps):
    optimizer.zero_grad()              # supplied
    prediction = ...                   # your function
    loss = ...                         # your MSE expression
    loss.backward()                    # supplied
    optimizer.step()                   # supplied
```

Run training and the plots. Compare the initial and trained curves and inspect the loss history. Does the trained network pass through every observation? Is this necessary?

(b) Evaluate on the 200 supplied fresh validation observations. Compare both MSEs. Which observations influenced the updates? Why can the errors differ?

(c) Explain the roles of the noisy observations, the true regression function, the initial network, and the trained network. Would passing through every training point guarantee good predictions on new observations?

Homework

Reuse the notebook and explain what the results mean

Deadline: 29 October 2026 **Groups:** at most four students

A. Change the regression function

Keep the input distribution and noise standard deviation, but replace the response model by

$$Y = |X| + \varepsilon.$$

(a) Complete the response expression in the supplied homework cell. The notebook supplies fresh initialization and the same training and evaluation procedure, reusing your `predict` function. Generate a fit plot and report the training and validation MSE. Describe where the trained curve follows the underlying function well and where discrepancies remain. Use `torch.abs(x)` for the entrywise absolute value.

(b) Prove that

$$|x| = \mathrm{ReLU}(x) + \mathrm{ReLU}(-x), \qquad x \in \mathbb{R}.$$

Give values of W_1, b_1, W_2, b_2 for a two-hidden-neuron network that represents $|x|$ exactly.

(c) Explain why training a network on noisy observations need not recover this exact representation, even though such parameters exist.

B. What is the best possible prediction?

Return to $Y = X^2 + \varepsilon$. Let f be a fixed measurable predictor with finite squared risk

$$R(f) = \mathbb{E}[(f(X) - Y)^2].$$

(a) Show that

$$R(f) = 0.01 + \mathbb{E}[(f(X) - X^2)^2].$$

Hint. Expand the square. Use independence and the fact that the noise has mean zero.

(b) Identify an optimal predictor and the minimum population risk. Explain the meaning of the two terms in the decomposition.

(c) Could a training or validation MSE computed from finitely many observations be smaller than 0.01? Explain why this would not contradict the identity.

Optional: the best affine predictor

Find the coefficients a, b minimizing $R(f_{a,b})$ for $f_{a,b}(x) = ax + b$. You may use

$$\mathbb{E}[X] = \mathbb{E}[X^3] = 0, \qquad \mathbb{E}[X^2] = \frac{1}{3}, \qquad \mathbb{E}[X^4] = \frac{1}{5}.$$

Compute its risk and explain why an affine predictor cannot achieve the optimal risk for this regression problem.

Submit: one executed notebook per group, headed **Group name:**, with your completed code, figures, numerical results, and mathematical answers. Restart the kernel and run all cells before submission. Follow the course instructions for the upload location and any PDF export. The optional problem is not required.